МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ "ЛЬВІВСЬКА ПОЛІТЕХНІКА"

Інститут **ІКНІ**

Кафедра **СШІ**

ЗВІТ

до лабораторної роботи № 1

з дисципліни: “Захист Інформації”
на тему: “СТВОРЕННЯ ТА ТЕСТУВАННЯ ГЕНЕРАТОРА ПСЕВДОВИПАДКОВИХ ЧИСЕЛ”

Лектор:
проф. каф. СШІ
Яковина В.С.

Виконав:
ст. гр. ШІ-23
Федів А. С.
.

18 лютого 2026 р.

Львів – 2026

Тема роботи: СТВОРЕННЯ ТА ТЕСТУВАННЯ ГЕНЕРАТОРА ПСЕВДОВИПАДКОВИХ ЧИСЕЛ.

Мета роботи: ознайомитись з джерелами та застосуванням випадковихчисел, алгоритмами генерування псевдовипадкових чисел та навчитисьстворювати програмні генератори псевдовипадкових чисел для використання в системах захисту інформації.

ХІД РОБОТИ

Я використовую Python (FastApi) + Typescript (React), як фронтенд та бекенд.
Для цієї лабораторної роботи мені я написав random_service.py який має такі методи:

```python
    @staticmethod
    def generate(config: GenConfig, n: int) -> list[int]:
        x = config.x0
        values = []
        for _ in range(n):
            x = (config.a * x + config.c) % config.m
            values.append(x)
        return values
```

Мал. 1. функція генерації масиву розміру n чисел методом лінійного порівняння

```python
@staticmethod
def chizaru(config: GenConfig, n: int) -> PiEstimateResponse:
    values = RandomService.generate(config, 2 * n)
    coprime_count = 0
    total_pairs = 0

    for i in range(0, len(values) - 1, 2):
        total_pairs += 1
        if RandomService.gcd_euclid(values[i], values[i + 1]) == 1:
            coprime_count += 1

    if coprime_count == 0:
        pi_estimate = float("inf")
    else:
        pi_estimate = math.sqrt((6.0 * total_pairs) / coprime_count)

    return PiEstimateResponse(
        pi_estimate=pi_estimate,
        abs_error_vs_math_pi=abs(pi_estimate - math.pi) if math.isfinite(pi_estimate) else float("inf"),
    )
```

Мал. 2. функція перевірки методом Чезаро на випадковість функції генерації випадкових чисел

```python
def gcd_euclid(a: int, b: int) -> int:
    a = abs(a)
    b = abs(b)
    while b != 0:
        a, b = b, a % b
    return a
```

Мал. 3. функція GCD, методом Евкліда

```python
def test_01_amount(gen_values: Callable[[int], list[int]], n: int) -> BitBalanceResponse:
    zeros = 0
    ones = 0

    for xi in gen_values(n):
        if xi == 0:
            zeros += 1
            continue

        while xi > 0:
            if xi % 2 == 0:
                zeros += 1
            else:
                ones += 1
            xi //= 2

    ratio = zeros / ones if ones > 0 else float("inf")
    return BitBalanceResponse(zeros=zeros, ones=ones, ratio_zero_to_one=ratio)
```

Мал. 4. Функція перевірки кількості бітів 0 та 1 у випадкових чисел

```python
@staticmethod
def sequence_period(config: GenConfig) -> SequencePeriodResponse:
    seen: set[int] = set()
    prev = config.x0
    steps = 0

    while steps < 1_000_000:
        xi = RandomService.generate(GenConfig(x0=prev, m=config.m, a=config.a, c=config.c), 1)[0]
        if xi in seen:
            return SequencePeriodResponse(period=steps, reached_limit=False)

        seen.add(xi)
        prev = xi
        steps += 1

    return SequencePeriodResponse(period=steps, reached_limit=True)
```

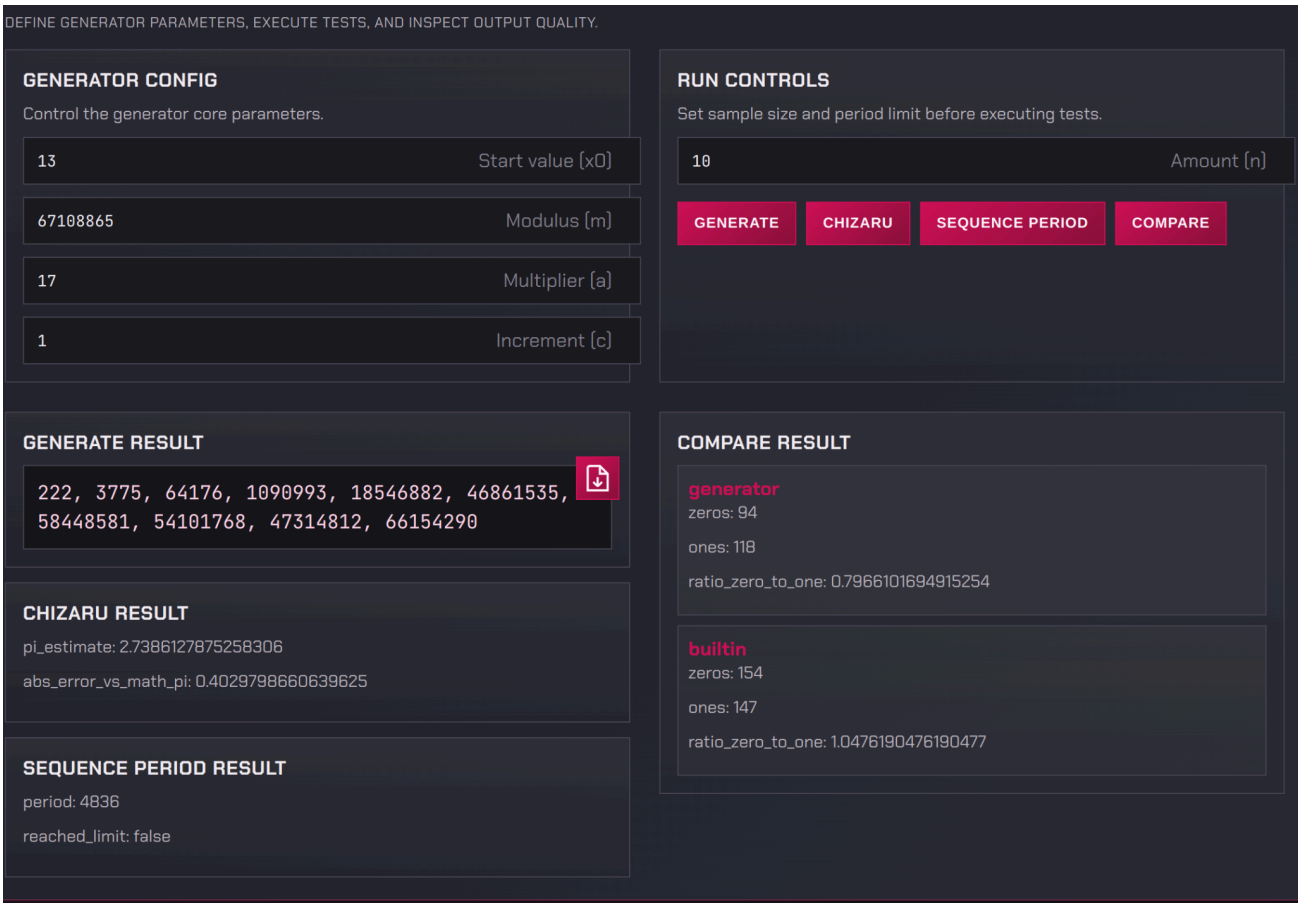
Мал. 5. Функція обчислення періоду генератора

```python
@staticmethod
def compare_with_builtin(config: GenConfig, n: int) -> CompareResponse:
    generator_stats = RandomService.test_01_amount(lambda count: RandomService.generate(config, count), n)

    random.seed(config.x0)
    builtin_stats = RandomService.test_01_amount(
        lambda count: [random.randint(0, 2**31 - 1) for _ in range(count)],
        n,
    )

    return CompareResponse(generator=generator_stats, builtin=builtin_stats)
```

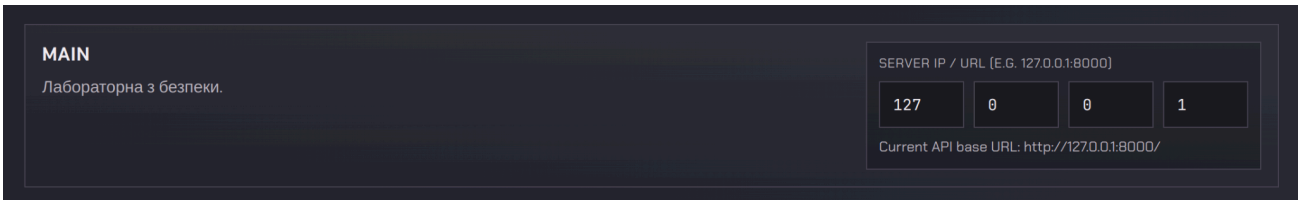
Мал. 6. Функція порівняння власного генератора з вбудованим на основі тесту за допомогою функції з мал. 4.

Також до кожного з цих методів, я зробив контролер та типізацію вхідних та вихідних даних. Використовую сервіс-контролерну архітектуру відповідно.

Фронтенд спілкуючись з бекендом, має можливість взаємодіяти з його API та дозволяє користувачеві, не вдаючись в деталі реалізації спробувати підібрати коефіцієнти для генератора (початково та за замовчуванням це є числа з мого варіанту лр), згенерувати масив розміром n випадкових чисел, та протестувати за різними параметрами якість генератора. Також я вирішив додати можливість ввести ip сервера, на випадок, якщо веб сторінку потрібно буде запустити на іншому пристрої.



Мал. 7. Сторінка з генератором та отриманими згенерованими числами та результатами вимірювань



Мал. 8. Основна сторінка з можливостями задати IP-адресу

Також однією з вимог була можливість запускати код не з IDE. Для цього я попросив GPT згенерувати .sh скрипт, який буде запускати стандартну збірку фронтенда на по одному статичному .html .js та .css файлу, а python сервіс буде їх видавати за / посиланням. Також в майбутньому я фронтенд засуну на Vercel і він буде завжди доступний в інтернеті.

КОНТРОЛЬНІ ЗАПИТАННЯ

1. Назвіть вимоги до послідовності псевдовипадкових чисел.
Послідовність повинна відповідати двом основним вимогам: випадковість (однорідність розподілу та незалежність елементів) і непередбачуваність (неможливість визначити наступні елементи за попередніми).

2. Назвіть вимоги щодо випадковості послідовності псевдовипадкових чисел.
Випадковість включає дві складові: однорідність розподілу — частота появи одиниць і нулів має бути приблизно однаковою; та незалежність — жодне значення послідовності не повинно виводитися з інших значень.

3. Що таке пряма і зворотна непередбачуваність послідовності псевдовипадкових чисел?
Пряма непередбачуваність означає, що наступний біт послідовності неможливо передбачити, навіть знаючи всі попередні, якщо затравка невідома. Зворотна непередбачуваність означає неможливість визначити значення затравки за будь-якими згенерованими значеннями.

4. Які властивості послідовності досліджують тести згідно NIST SP 800-22?
Стандарт визначає три властивості: однорідність (ймовірність появи одиниць і нулів дорівнює ½), масштабованість (будь-яка підпослідовність також має бути випадковою) та стійкість (поведінка генератора має бути стабільною при різних значеннях затравки).

5. Що таке генератор псевдовипадкових чисел?
Це алгоритм, що генерує послідовність чисел, елементи якої статистично незалежні один від одного і підлягають заданому розподілу. На відміну від істинно випадкових чисел, послідовність генерується детермінованим способом.

6. Що таке період послідовності псевдовипадкових чисел?
Період — це кількість елементів у послідовності до першого повторення.. Ідеальний генератор повного періоду породжує всі числа перед повторенням.

7. Назвіть критерії оцінки якості генератора псевдовипадкових чисел.
Три основні критерії: функція генерації повинна бути функцією повного періоду; створена послідовність повинна поводитись статистично як випадкова; функція генерації повинна ефективно реалізовуватись у 32-бітній арифметиці.

8. Якими повинні бути параметри алгоритму лінійного порівняння для генерації повного періоду?
Якщо m є простим числом і $c = 0$, то для певних значень a період генерованої послідовності дорівнює $m-1$. Зручним значенням є $m = 2^{31}-1$, а одним із відповідних множників — $a = 7^5 = 16807$.

9. Назвіть переваги алгоритму лінійного порівняння.
За умови адекватного вибору множника та модуля порівняння, послідовність виявляється статистично невідрізнимою від справді випадкової. Алгоритм простий, ефективний і добре реалізується у 32-бітній арифметиці.

10. Назвіть недоліки алгоритму лінійного порівняння.
Послідовність є повністю детермінованою: знаючи параметри алгоритму та хоча б одне число послідовності, можна відновити всі наступні. Зі знанням чотирьох елементів можна визначити всі параметри a, c, m і відтворити послідовність.

ВИСНОВКИ

На цій лабораторній роботі я ознайомився з основами генерування псевдовипадкових чисел та їх застосуванням у системах захисту інформації. Я реалізував генератор псевдовипадкових чисел за алгоритмом лінійного порівняння Лемера з параметрами згідно мого варіанту, а також набір функцій для тестування якості генератора.

Практично переконався, що вибір параметрів алгоритму (m, a, c, X_0) є критично важливим — при невдалому виборі послідовність має малий період і очевидно не є придатною для криптографічних задач. Перевірка періоду генератора підтвердила, що при правильно підібраних параметрах він наближається до максимально можливого значення $m-1$.

Тестування за методом Чезаро дозволило статистично оцінити якість генератора через наближення значення числа π. Чим більша кількість згенерованих чисел використовується у тесті, тим точнішим виявляється результат, що свідчить про достатній рівень випадковості послідовності. Порівняння власного генератора з вбудованим генератором Python показало, що за співвідношенням нулів і одиниць у бітовому представленні обидва генератори дають схожі результати.

Водночас я переконався у головному недоліку алгоритму лінійного порівняння — його повній детермінованості. Знаючи параметри та хоча б одне значення послідовності, можна відтворити всі наступні елементи, що робить такий генератор непридатним для використання як єдиного засобу захисту у криптографічних системах без додаткових заходів, як-от модифікація потоку за допомогою системного часу.

З технічної сторони я отримав практичний досвід побудови повноцінного вебзастосунку з клієнт-серверною архітектурою (FastAPI + React), що дозволяє зручно взаємодіяти з реалізованими алгоритмами через графічний інтерфейс без необхідності заглиблюватись у деталі реалізації.